SMART CITY TRAFFIC MANAGEMENT AND ROUTE OPTIMIZATION SYSTEM



School of Engineering & Technology

K.R. Mangalam University

DAA LAB CAPSTONE PROJECT

Course: Design and Analysis of Algorithms Lab (ENCA351)

Program: BCA (AI & DS) | Semester: V | Session: 2026–27

Submitted by: Kshitij

Roll No.: 2401201021

Submitted to: Dr. Aarti

1. Introduction

Smart cities use digital infrastructure to improve urban services including transportation. Efficient traffic management reduces travel time, lowers fuel consumption, and helps emergency services reach their destinations faster. A core problem in traffic management is finding optimal routes between locations and identifying which roads form the essential backbone of the network.

A city road network maps naturally to a weighted undirected graph. Each intersection becomes a vertex and each road between two intersections becomes an edge. The weight of an edge represents the travel time on that road, measured in minutes. This representation lets us apply classical graph algorithms to answer practical questions: what is the fastest route from A to B, what happens when a road is congested, which intersections are reachable, and what is the minimum set of roads needed to keep the entire city connected.

Project objectives

- Model the city road network as a weighted undirected graph.
- Implement Dijkstra's algorithm to find shortest travel-time routes.
- Use Bellman-Ford to simulate congestion and negative-weight scenarios.
- Apply BFS and DFS to analyse network connectivity and reachability.
- Use Prim and Kruskal to find minimum-cost spanning trees for infrastructure planning.
- Compare algorithm performance across different network sizes.
- Present all results in an interactive Streamlit web application.

2. Methodology

Road network model (road_network.py)

The RoadNetwork class stores the graph as a dict of dicts: self.adj[u][v] = weight. Python dicts preserve insertion order, so node and neighbour order is deterministic. Methods cover adding, updating, and removing intersections and roads, plus tabular exports as adjacency list and adjacency matrix.

Dijkstra's algorithm (route_planner.py)

Dijkstra finds the shortest path from a source to all other nodes using a binary min-heap (heapq). Time complexity: O((V + E) log V). The algorithm is correct for graphs with non-negative edge weights, which is guaranteed because all road weights are positive travel times.

Bellman-Ford algorithm (traffic_analysis.py)

Bellman-Ford relaxes all edges up to V - 1 times and stops early when no update occurs in a full pass. Time complexity: O(V * E). It detects negative-weight cycles in a final extra pass. For this project it is used to model congestion by multiplying road weights.

BFS and DFS (connectivity_module.py)

BFS uses a deque and visits nodes level by level; DFS uses an explicit stack and visits nodes depth-first. Both have time complexity O(V + E). They are used to check reachability and report connected components.

Prim and Kruskal (mst_planner.py)

Prim grows the MST outward from a start node using a min-heap: O(E log V). Kruskal sorts edges by weight and uses Union-Find with path compression and union by rank to build the MST: O(E log E). Both produce a spanning forest on disconnected graphs.

Algorithm complexity summary

Table 1: Algorithm time complexities

Algorithm	Time complexity	Best used for
BFS	O(V + E)	Fewest roads and level-by-level exploration.
DFS	O(V + E)	Deep exploration and reachability checks.
Dijkstra (binary heap)	O((V + E) log V)	Fast routing with non-negative travel costs.
Bellman-Ford	O(V * E)	Networks that may contain negative adjustments, and detecting negative cycles.
Prim (binary heap)	O(E log V)	Connected networks, grows one tree outward.
Kruskal	O(E log E)	Sparse networks, works edge

		by edge and handles forests.

3. Experimental results

Sample road network

The sample city has 10 intersections and 16 roads. The network is connected.

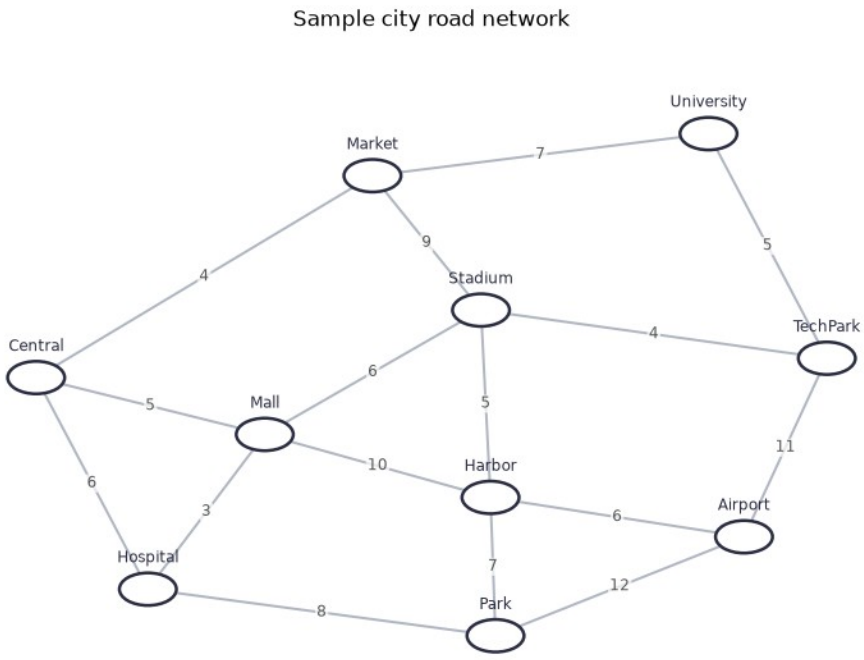


Figure 1: Sample city road network

Adjacency list (first five intersections)

Table 2: Adjacency list (first five intersections)

Intersection	Connected roads
Central	Market (4), Hospital (6), Mall (5)
Market	Central (4), University (7), Stadium (9)
Hospital	Central (6), Mall (3), Park (8)
Mall	Central (5), Hospital (3), Stadium (6), Harbor (10)
University	Market (7), TechPark (5)

Dijkstra shortest route

Shortest route from Central to Airport: Central -> Mall -> Harbor -> Airport. Total cost: 21 min.

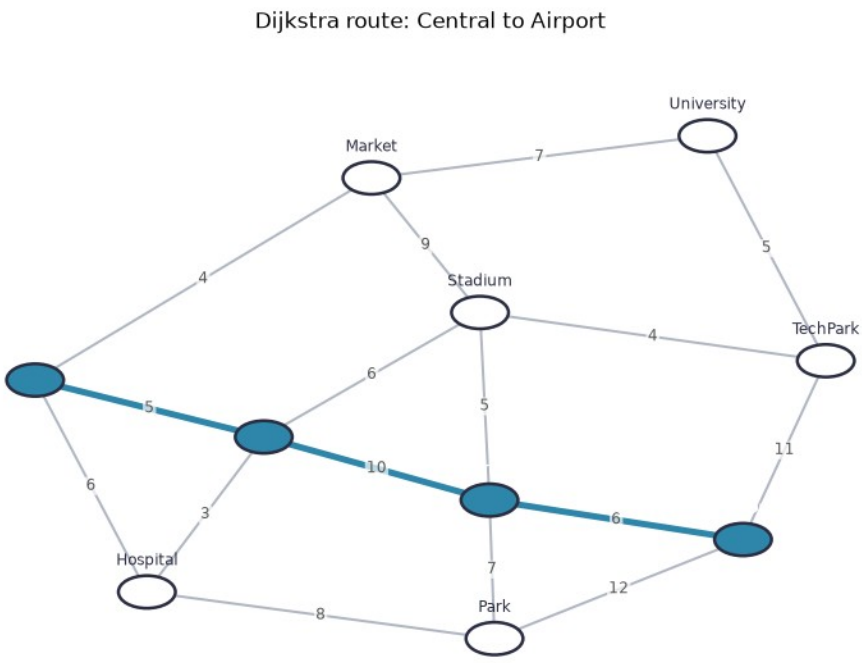


Figure 2: Dijkstra shortest route: Central to Airport

Traffic analysis: congestion on Mall-Harbor (x3)

Original route: Central -> Mall -> Harbor -> Airport, cost 21 min. After multiplying Mall-Harbor weight by 3: Central -> Mall -> Stadium -> Harbor -> Airport, cost 22 min. Cost change: +1 min.

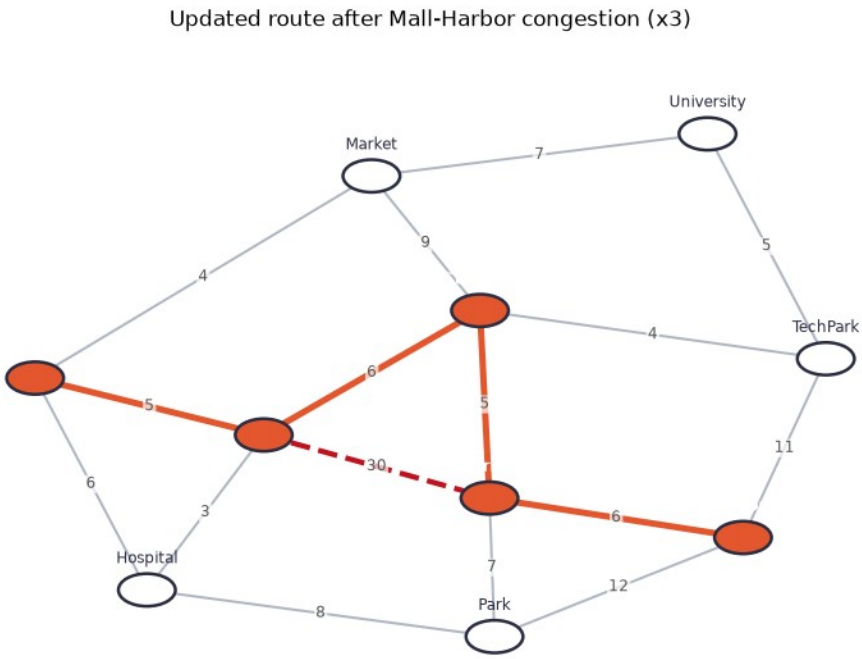


Figure 3: Updated route after Mall-Harbor congestion

BFS traversal from Central

BFS order: Central > Market > Hospital > Mall > University > Stadium > Park > Harbor > TechPark > Airport

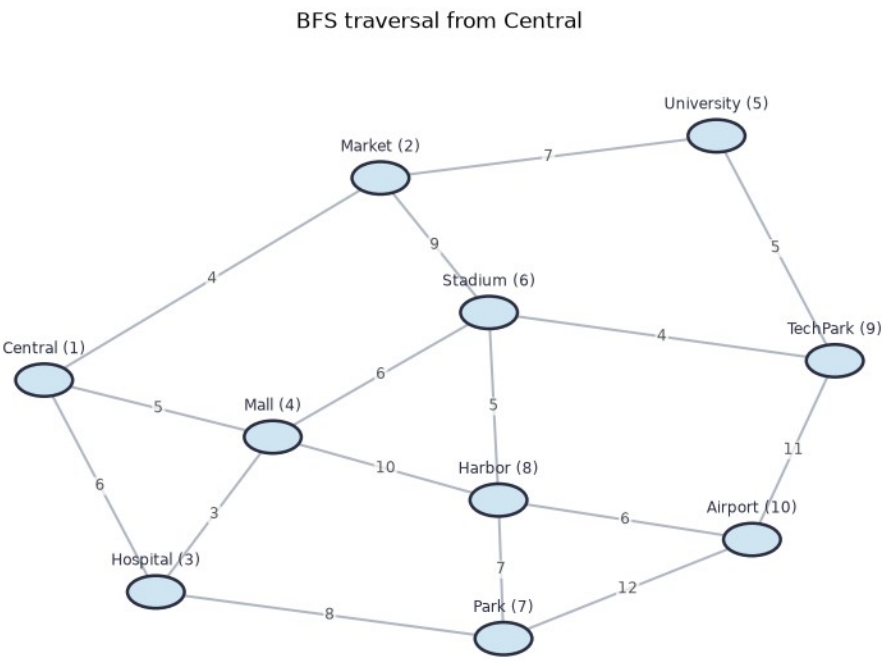


Figure 4: BFS traversal from Central

DFS traversal from Central

DFS order: Central > Market > University > TechPark > Stadium > Mall > Hospital > Park > Harbor > Airport

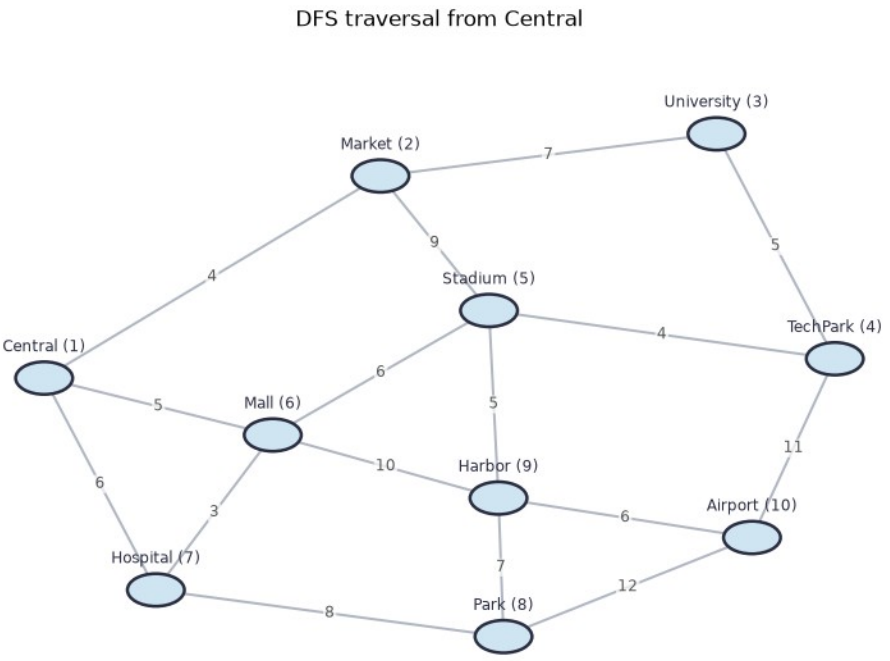


Figure 5: DFS traversal from Central

Prim minimum spanning tree

Table 3: Prim MST roads (total: 45 min)

From	To	Cost (min)
Central	Market	4.0
Central	Mall	5.0
Mall	Hospital	3.0
Mall	Stadium	6.0
Stadium	TechPark	4.0
Stadium	Harbor	5.0
TechPark	University	5.0
Harbor	Airport	6.0
Harbor	Park	7.0

Kruskal minimum spanning tree

Table 4: Kruskal MST roads (total: 45 min)

From	To	Cost (min)
Hospital	Mall	3.0
Central	Market	4.0
Stadium	TechPark	4.0

Central	Mall	5.0
University	TechPark	5.0
Stadium	Harbor	5.0
Mall	Stadium	6.0
Harbor	Airport	6.0
Park	Harbor	7.0

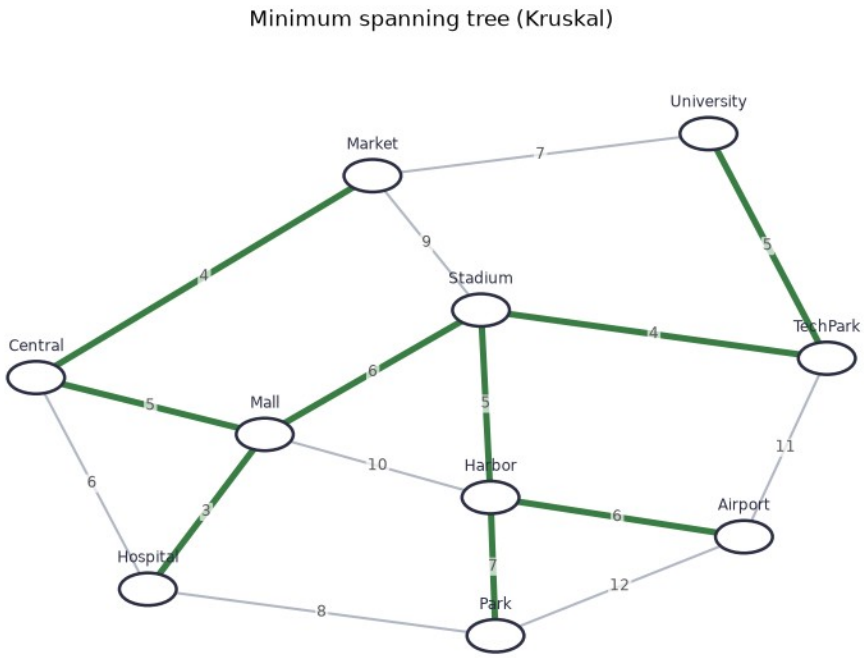


Figure 6: Minimum spanning tree (Kruskal)

Benchmark results

Table 5: BFS vs DFS benchmark

Intersections	Roads	BFS time (ms)	DFS time (ms)	BFS visited	DFS visited	BFS memory (KB)	DFS memory (KB)
10.0	15.0	0.0065	0.0105	10.0	10.0	1.8	1.5
25.0	37.0	0.0155	0.0267	25.0	25.0	3.8	3.3
50.0	75.0	0.0315	0.0538	50.0	50.0	3.7	3.5
100.0	150.0	0.0652	0.1162	100.0	100.0	12.1	11.6

Table 6: Dijkstra vs Bellman-Ford benchmark

Interse	Roads	Dijkstr	Bellma	Dijkstr	Bellma	Route	Bellma	Dijkstr	Bellma

ctions		a time (ms)	n-Ford time (ms)	a cost	n-Ford cost	hops	n-Ford passes	a memory (KB)	n-Ford memory (KB)
10.0	15.0	0.0157	0.034	27.0	27.0	2.0	2.0	1.5	1.0
25.0	37.0	0.0367	0.1009	22.0	22.0	4.0	3.0	4.4	3.2
50.0	75.0	0.0812	0.211	22.0	22.0	6.0	3.0	6.7	5.9
100.0	150.0	0.1773	0.5953	37.0	37.0	3.0	5.0	18.4	12.2

Table 7: Prim vs Kruskal benchmark

Intersections	Roads	Prim time (ms)	Kruskal time (ms)	Prim cost	Kruskal cost	MST roads	Prim memory (KB)	Kruskal memory (KB)
10.0	15.0	0.013	0.0189	49.0	49.0	9.0	1.4	1.0
25.0	37.0	0.033	0.0434	177.0	177.0	24.0	3.7	3.2
50.0	75.0	0.0687	0.0979	327.0	327.0	49.0	4.2	5.9
100.0	150.0	0.1636	0.1843	787.0	787.0	99.0	13.2	12.2

Application screenshots

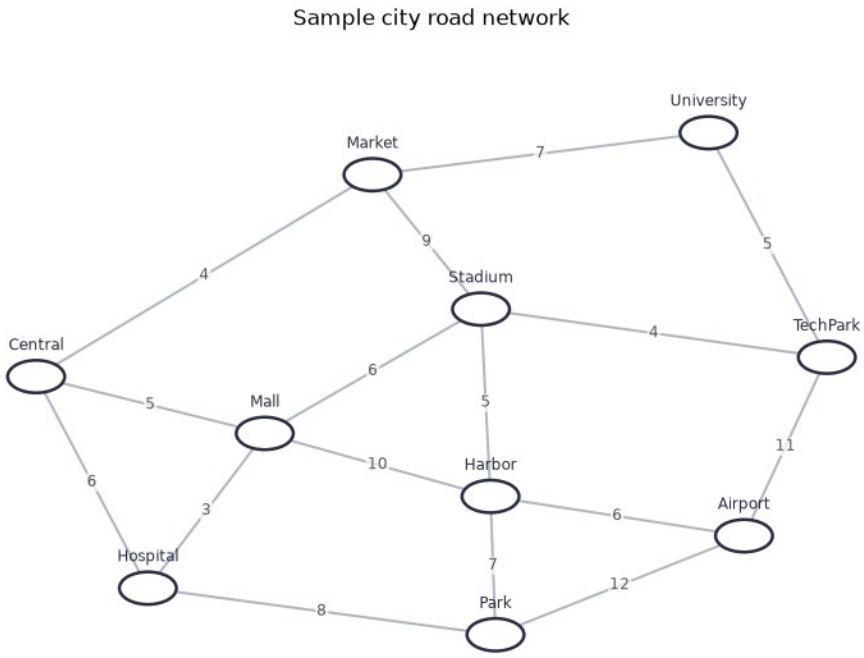


Figure 7: Road network page

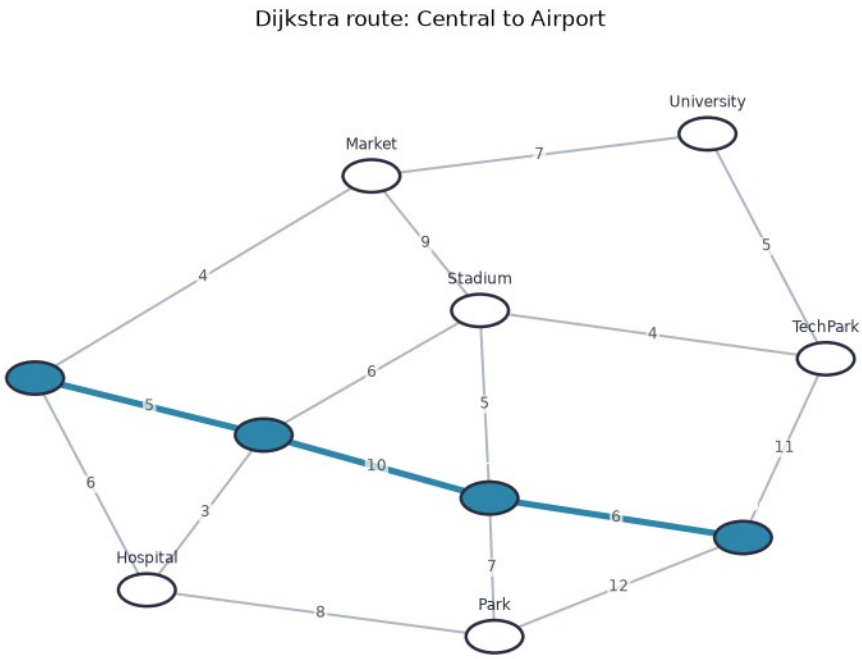


Figure 8: Route planning page

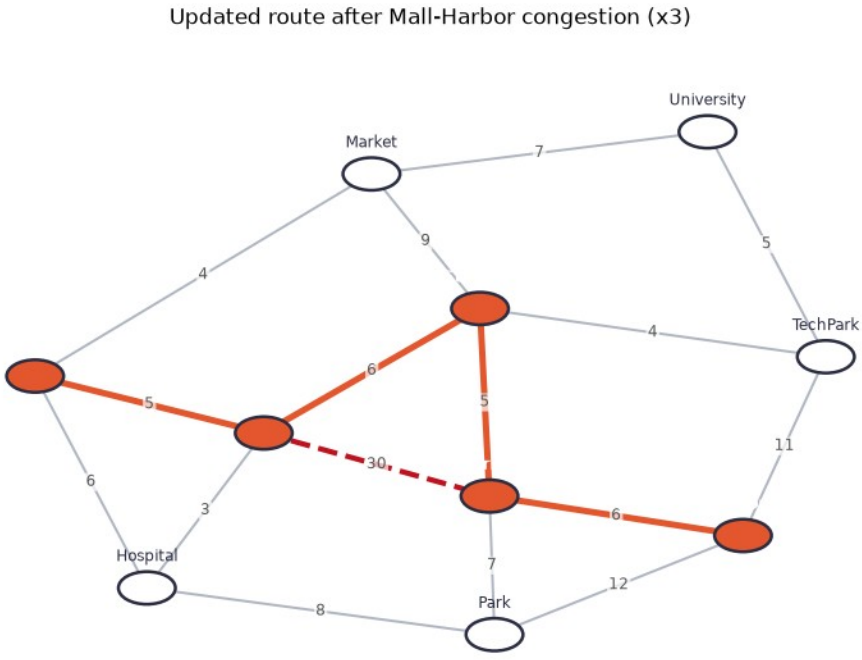


Figure 9: Traffic analysis page

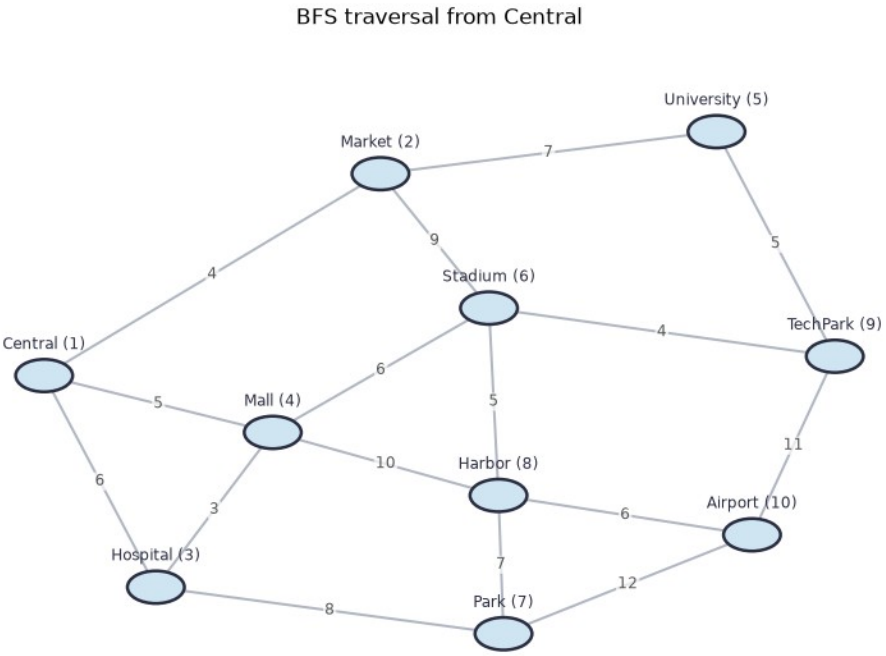


Figure 10: Network connectivity page

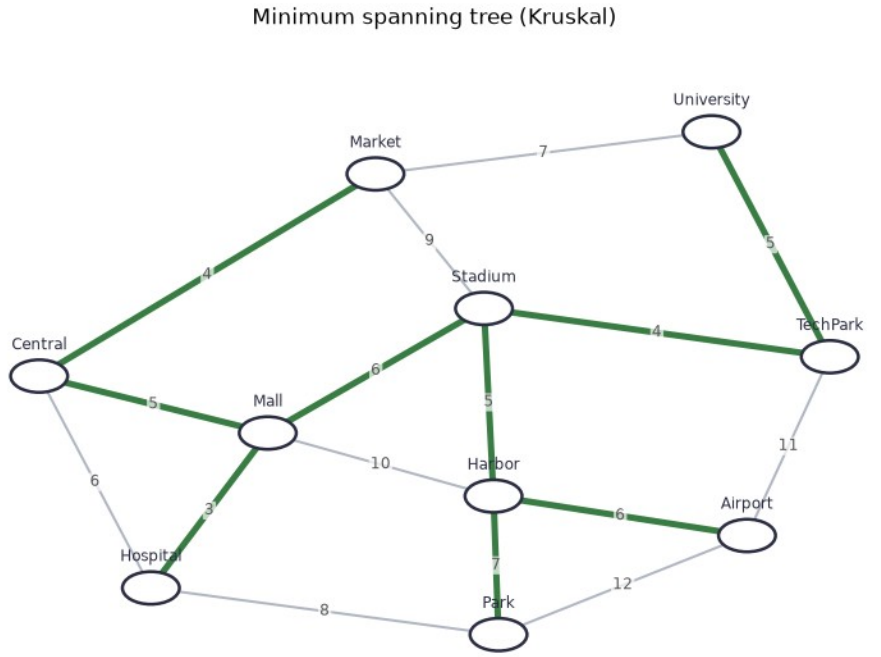


Figure 11: Infrastructure planning page

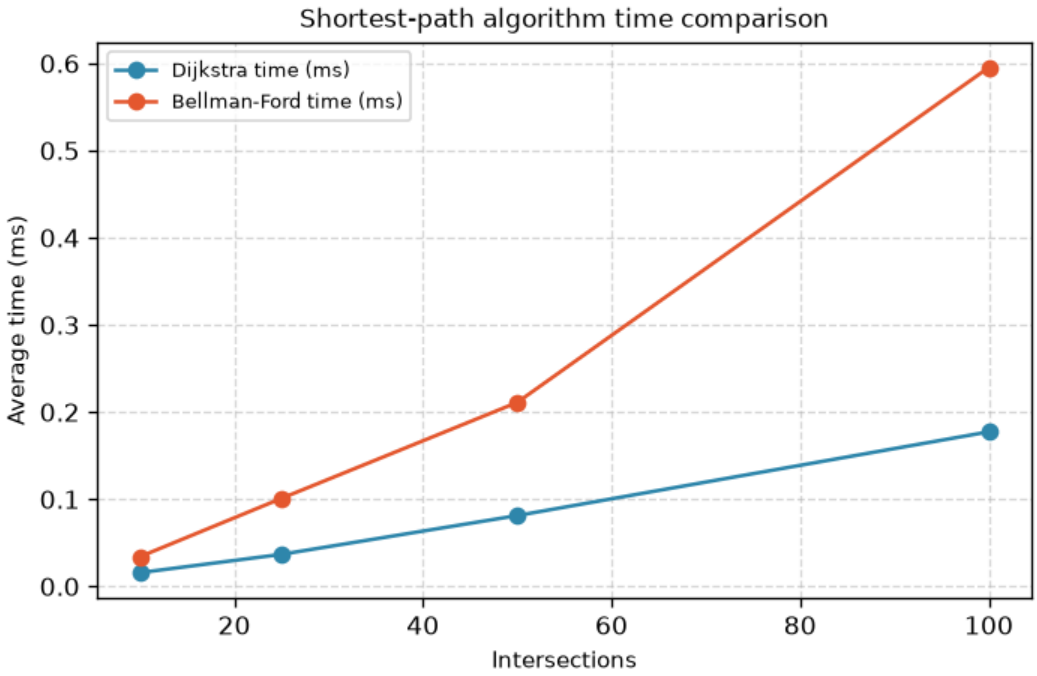


Figure 12: Comparative analysis page

4. Analysis

BFS vs DFS

Both BFS and DFS have O(V + E) time complexity, so their raw runtimes are similar. BFS visits nodes in order of distance from the source (measured in hops), which makes it useful for finding the route with the fewest roads. DFS follows one branch as far as possible before backtracking, which is useful for reachability checks and detecting cycles. For route planning in a weighted network, neither algorithm is the right tool because they do not consider edge weights.

Dijkstra vs Bellman-Ford

Dijkstra's algorithm is faster in practice because it settles each node once and processes only the most promising node at each step. Its O((V + E) log V) complexity makes it significantly faster than Bellman-Ford's O(V * E) on larger networks. The benchmark tables confirm this: Dijkstra's time grows much more slowly than Bellman-Ford's as network size increases. Bellman-Ford's advantage is that it can handle negative edge weights and detect negative cycles, which Dijkstra cannot.

Prim vs Kruskal

Prim and Kruskal produce the same MST total cost on connected networks, as confirmed by both giving 45 min on the sample city. Prim grows one tree from a start node, which suits dense networks. Kruskal sorts all edges and picks them in order, which handles disconnected networks naturally by producing a spanning forest.

5. Conclusion

This project demonstrated that classical graph algorithms provide practical solutions to real traffic management problems. Dijkstra found the optimal route from Central to Airport in 21 min, and Bellman-Ford confirmed the same result while also showing how congestion changes redirect traffic. Prim and Kruskal both identified the minimum spanning tree at 45 min, giving city planners the cheapest connected road layout. BFS and DFS confirmed that the sample city network is fully connected.

Future improvements

- Support directed one-way roads for more realistic city modelling.
- Integrate live traffic data feeds to update edge weights in real time.
- Implement A* search for faster point-to-point routing on large maps.
- Use real map data from OpenStreetMap instead of a hand-crafted sample city.
- Add traffic signal timing optimisation to the intersection model.

6. References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. Introduction to Algorithms, 4th edition. MIT Press, 2022.
2. NetworkX documentation. https://networkx.org/documentation/stable/
3. Streamlit documentation. https://docs.streamlit.io/
4. Matplotlib documentation. https://matplotlib.org/stable/contents.html
5. Pandas documentation. https://pandas.pydata.org/docs/